Cantina Discovery Pipeline

AI GTM Engineer, Take Home Submission

Prashanth T

March 2026

1. Hypothesis Selection

I chose Hypothesis A: "Your security stack wasn't built for AI-generated code."

Urgency. In Web3, a single smart contract vulnerability means immediate, irreversible loss of funds. Balancer lost $100M from a rounding error. Cetus DEX lost $223M from a missing overflow check. $3.1B was lost across Web3 in H1 2025 alone. Hypothesis A addresses an existential risk. Protocols either secure AI-generated code or face catastrophic loss. Hypothesis B (tool consolidation) is an efficiency problem. It's real, but it doesn't create the same pressure to act now.

The ICP is observable from public data. An automated pipeline can be built against signals that are fully accessible: TVL from DeFiLlama, audit history from bounty platforms, GitHub commit velocity, AI tool config files in repos. Hypothesis B requires knowing what internal security tools a team runs. That data is invisible from outside. An automated signal pipeline needs data it can actually reach.

The hypothesis extends beyond Web3. The same dynamic is playing out in Web2. 45% of AI-generated code contains security vulnerabilities (Veracode 2025, 100+ LLMs tested), and that rate hasn't improved as models get smarter. Engineering teams across SaaS, fintech, and enterprise are shipping AI-assisted code faster than their security processes can review it. The pipeline architecture is modular. The same scoring engine, outreach agent, CRM integration, and feedback loop work for Web2 targets by swapping the data sources (DeFiLlama becomes Crunchbase, TVL becomes ARR, bounty platforms become compliance databases) and adjusting the scoring weights. Web3 is the starting point because the data is public and the stakes are highest, but the system is designed to scale across both.

2. Signal Pipeline

The pipeline runs in four stages.

Find. DeFiLlama is the entry point. It exposes every DeFi protocol with TVL, category, and chain data via a public API. No scraping needed. The pipeline filters by category (DEXes, Lending, Yield, Bridges, Liquid Staking, RWA, Derivatives, Restaking) and minimum TVL ($50M), which narrows the funnel to protocols that have real funds at risk and a realistic security budget.

Enrich. For each protocol, the pipeline pulls three additional signal layers. GitHub: find the org, measure commit velocity over the last 30 days, check for AI tool config files (.cursorrules, .github/copilot-instructions.md), and flag repos with active Solidity or Rust development. Audit history: check whether the protocol has been audited, by whom, how recently, and whether significant new code has shipped since the last review. Funding: surface recent raises, which signal available budget for security services. Then comes contact discovery. The production approach would be Apollo.io, which takes a company domain and returns employees with titles, emails, and LinkedIn URLs, ideal for finding founders, CTOs, and security leads at scale. Since Apollo's API requires a paid plan, the pipeline uses two free alternatives that produce reliable results: GitHub contributor data (names, emails, and profiles of developers who committed code in the last 30 days, verified active by definition) combined with Claude web search (searches the live web to find current leadership, roles, and Twitter handles from recent articles and protocol websites). Between these two sources, the pipeline covers both engineering contacts and executive/security leadership without any paid enrichment subscriptions.

Score. Every protocol gets a 100-point composite score across five signals: TVL and funds at risk (30 pts), audit status (25 pts), shipping velocity (20 pts), funding recency (15 pts), and reachability (10 pts). The weights and thresholds live in a JSON config file, so the model can be recalibrated after discovery calls without touching code. Protocols scoring 75 or above qualify as warm leads. 90 or above is hot.

Act. Qualified leads go to Claude API, which generates a personalized cold email per contact using the enrichment data, referencing their specific TVL, auditor names, bounty amount, and the vulnerability class most relevant to their protocol category (DEX → reentrancy, lending → oracle manipulation, yield → vault accounting errors, bridge → cross-chain validation). Emails are delivered via Resend, contacts and companies are pushed to HubSpot with full signal data attached, and everything is persisted to PostgreSQL. A Slack alert fires for every pipeline run and for any hot lead. The pipeline runs from a Next.js UI. One click to run, results visible immediately.

The key design decision was making every threshold config-driven. After 10 discovery calls, if audit status predicts conversion better than TVL, two numbers change in a JSON file and the next run reflects it. The pipeline gets smarter with every batch of calls without a single code change.

3. Analysis

The pipeline scored 50 live Web3 protocols from DeFiLlama. 5 qualified as warm leads (score 75 to 89), 1 scored hot (≥ 90). The remaining 44 scored cool, mostly because they lacked GitHub activity data or had inactive development, which zeroed out their velocity score.

What the data shows. Every single qualified target in the pipeline has shipped new unaudited contract code since their last security review. That's the core Hypothesis A prediction made concrete. Development velocity is outpacing security coverage across the board. It's not an edge case, it's the norm for active protocols.

The bounty-to-TVL ratios stood out most. Ethena has a $750K bug bounty protecting $5.2B in TVL, that's 0.014%. EigenLayer runs $2M against $8B, 0.025%. Kamino Finance has $0 against $1.2B, no whitehat incentive at all. These aren't protocols ignoring security. They've been audited, they have active bounty programs, they're paying attention. But the continuous coverage relative to the funds at risk is thin.

What confirms the hypothesis. The audit supply-demand gap is real and measurable. Top audit firms have 2 to 3 month waitlists. Active protocols are shipping weekly. That gap is where exploits happen. $263M in smart contract losses in Q1 2025 alone, the worst quarter since early 2023. Balancer lost $100M to a rounding error. Cetus lost $223M to a missing overflow check. These are exactly the vulnerability classes AI-generated code is known to produce: arithmetic errors, missing access controls, reentrancy. The correlation is structural, not coincidental.

What challenges it. Most protocols scored cool (below 75) not because they're poor targets, but because the pipeline lacked enrichment data for them, specifically GitHub activity and audit history. With fuller data coverage, many of those 44 protocols would likely score higher. This is a pipeline coverage gap, not a hypothesis problem. In production, expanding GitHub org matching and integrating audit databases from major bounty and competition platforms would move more protocols into the qualified range.

The harder challenge is that AI-for-Solidity adoption is less visible than general AI coding adoption. Copilot and Cursor usage can be inferred from job posts and GitHub config files. But whether a specific smart contract was written with AI assistance is not something developers announce publicly. The hypothesis is structurally sound, but proving the AI angle specifically requires asking directly on discovery calls rather than inferring from external signals.

What I'd do differently. Add pre-launch protocols to the top of the funnel. A protocol that has raised money, has Solidity in GitHub, but hasn't deployed to mainnet yet is the highest-intent target. They need an audit before launch, they have budget, and they're on a deadline. That signal is fully detectable from public data and should be weighted heavily.

Second, instrument on-chain deployment events. Every time a pipeline target deploys a new contract to mainnet, that's a trigger. "The new code isn't audited yet" outreach sent within 24 hours. The monitoring infrastructure is built for this pattern, it just needs the event-to-outreach loop connected.

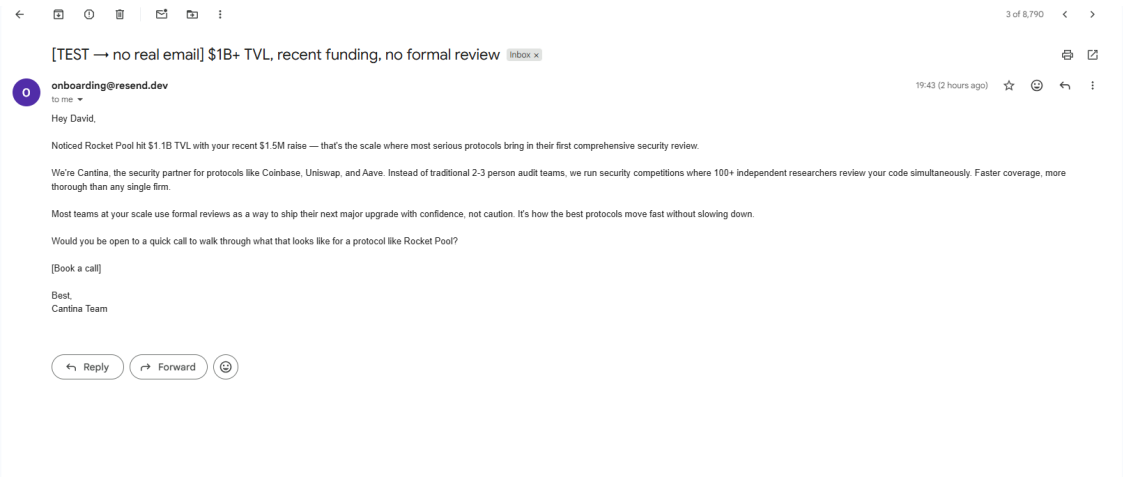
Third, after 10 calls, let the data drive weight adjustments. If audit status consistently predicts conversion

better than TVL, two numbers change in config/scoring_weights.json and the next run reflects it. No code changes needed. The pipeline reads weights from config on every run.

4. Outreach

The pipeline generates personalized outreach automatically. Claude API receives the full enrichment profile for each contact: TVL, audit history, bounty amount, protocol category, shipping velocity, and the person's role. It produces a unique email per person. Each message references signals specific to that protocol and uses a risk hook matched to their category. No templates are reused across protocols.

Below are three emails the pipeline generated and delivered to a test inbox via Resend.



3 of 8,790

[TEST → no real email] $1B+ TVL, recent funding, no formal review Inbox ×

onboarding@resend.dev 19:43 (2 hours ago)
to me

Hey David,

Noticed Rocket Pool hit $1.1B TVL with your recent $1.5M raise — that's the scale where most serious protocols bring in their first comprehensive security review.

We're Cantina, the security partner for protocols like Coinbase, Uniswap, and Aave. Instead of traditional 2-3 person audit teams, we run security competitions where 100+ independent researchers review your code simultaneously. Faster coverage, more thorough than any single firm.

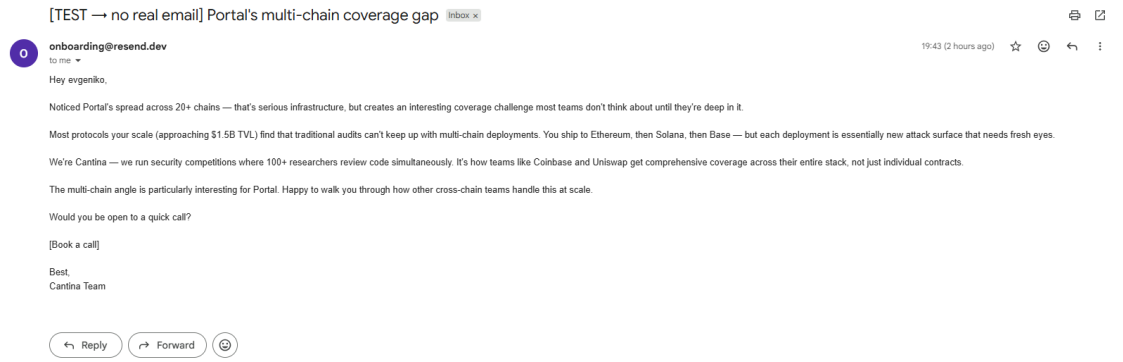
Most teams at your scale use formal reviews as a way to ship their next major upgrade with confidence, not caution. It's how the best protocols move fast without slowing down.

Would you be open to a quick call to walk through what that looks like for a protocol like Rocket Pool?

[Book a call]

Best,
Cantina Team

Reply Forward



[TEST → no real email] Portal's multi-chain coverage gap Inbox ×

onboarding@resend.dev 19:43 (2 hours ago)
to me

Hey evgeniko,

Noticed Portal's spread across 20+ chains — that's serious infrastructure, but creates an interesting coverage challenge most teams don't think about until they're deep in it.

Most protocols your scale (approaching $1.5B TVL) find that traditional audits can't keep up with multi-chain deployments. You ship to Ethereum, then Solana, then Base — but each deployment is essentially new attack surface that needs fresh eyes.

We're Cantina — we run security competitions where 100+ researchers review code simultaneously. It's how teams like Coinbase and Uniswap get comprehensive coverage across their entire stack, not just individual contracts.

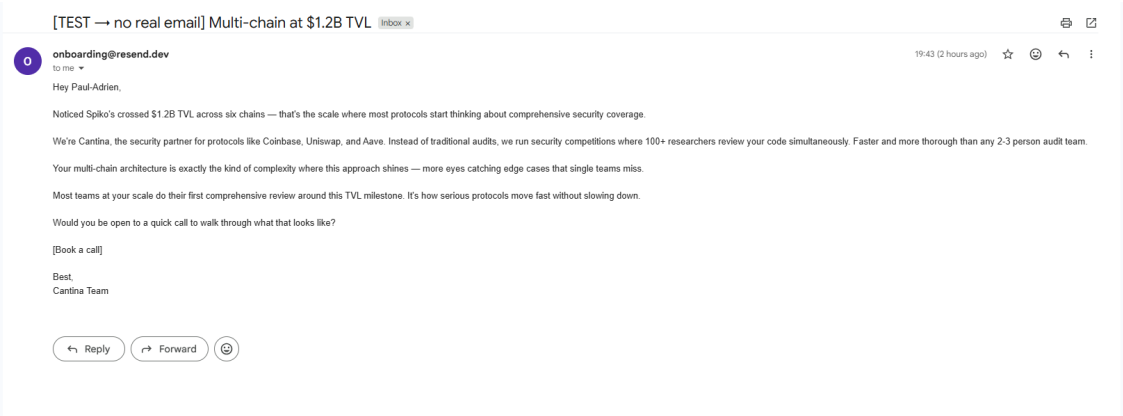
The multi-chain angle is particularly interesting for Portal. Happy to walk you through how other cross-chain teams handle this at scale.

Would you be open to a quick call?

[Book a call]

Best,
Cantina Team

Reply Forward



[TEST → no real email] Multi-chain at $1.2B TVL Inbox ×

onboarding@resend.dev 19:43 (2 hours ago)
to me

Hey Paul-Adrien,

Noticed Spiko's crossed $1.2B TVL across six chains — that's the scale where most protocols start thinking about comprehensive security coverage.

We're Cantina, the security partner for protocols like Coinbase, Uniswap, and Aave. Instead of traditional audits, we run security competitions where 100+ researchers review your code simultaneously. Faster and more thorough than any 2-3 person audit team.

Your multi-chain architecture is exactly the kind of complexity where this approach shines — more eyes catching edge cases that single teams miss.

Most teams at your scale do their first comprehensive review around this TVL milestone. It's how serious protocols move fast without slowing down.

Would you be open to a quick call to walk through what that looks like?

[Book a call]

Best,
Cantina Team

Reply Forward

Full drafts are persisted in PostgreSQL and exported to data/output/outreach_drafts.json.

5. Instrumentation

Every call feeds a structured post-call form. Two minutes to fill out, maps directly to the discovery_calls table in PostgreSQL. The fields that matter most:

- Did they confirm the pain? (Y/N)
- Pain severity, self-reported 1 to 10
- Are their devs using Copilot, Cursor, or Claude Code for Solidity? Which tools?
- Is their audit process keeping pace with their shipping speed? (Y/N)
- Have they had an exploit or near-miss?
- What's their current bounty platform?
- What did the enrichment data get wrong about them?
- What's the next step: scoping call, demo, CISO intro, or nothing?

At the pipeline level, the key metrics are booking rate (target >15%), pain severity average (target ≥7/10), hypothesis confirmation rate (target >60%), and next-step conversion (target >50%). Channel attribution as well. Whether Twitter DM, email, or warm intro is driving the bookings, because that determines where to invest outreach effort.

What confirmed signal looks like at Day 30.

The strongest confirmation is a prospect saying: "Yes, our devs are using Copilot for Solidity. Yes, we've found bugs in AI-generated contract code. Yes, our audit process can't keep up. What can Cantina do?"

But partial confirmation is equally useful. "We're not using AI for contracts yet, but our audit process is already too slow" means the velocity pain is real even without the AI angle. "We use a bounty platform but the submission quality is low" means Cantina's researcher quality and triage is the differentiator. "We can't get on an auditor's calendar for three months" means the supply-demand gap is the pain. Tracking the nuance matters more than tracking a binary yes/no.

The Day 30 decision.

6+ of 10 calls confirm AI code security gaps, pain severity ≥7/10 average, 3+ convert to a next step. That's the signal to scale. Double the pipeline targets, automate the full run on a daily cron, activate event-triggered outreach so a new contract deployment or exploit in the same category fires a contextual email within 24 hours.

4 to 5 confirm but severity is moderate. Refine. Check if the pain is sharper for specific categories (bridges and lending tend to have higher stakes than DEXes) or protocol stages (pre-launch vs established). Tighten the ICP before scaling.

Fewer than 4 confirm. Stop and diagnose. Wrong messaging, wrong audience, or wrong hypothesis. At that point, run 5 calls framed around Hypothesis B, tool consolidation, and compare pain severity directly.